Microsoft

Azure Cosmos DB for AI

AI has forever changed what software makes possible (and what we expect from it)

It's elevating our expectations

ChatGPT crossed **1 million users in 5 days** of launch, setting the platform record[1]

It's reshaping every industry

The value of AI is projected to **increase 13x**—to $15.7 trillion by 2030[2]

It's rejuvenating transformation

87% of organizations believe AI will give them a **competitive edge**[3]

[1]OpenAI public statements
[2]Global Artificial Intelligence Study: Exploiting the AI Revolution, PwC
[3]AI Global Executive Study and Research Project, MIT Sloan and BCG

AI has forever changed what software makes possible (but somethings remain the same)

Generative AI has the potential to create $2.6 to $4.4 trillion in value across industries and automate activities that absorb 60–70% of employees' time.

Apps are the centerpiece of our digital economy

83% of enterprise decision makers plan to increase AI spend this year[1]

Apps are where the power of AI and data come to life

[1]The economic potential of generative AI
McKinsey June 2023

What is an AI Agent?

"An **agent** is a system that uses an LLM to decide the control flow of an application" – LangGraph

- **Planning.** AI agents can create step-by-step plans and can use feedback to improve refine future outputs.
- **Memory.** Agents rely on short-term memory for immediate prompts and leverage long-term data retention—often via retrieval-augmented generation (RAG)—for broader context and recall.
- **Perception.** AI agents can retrieve data and process information making them more interactive and contextually aware.
- **Tool Use.** Agents can run functions or call external APIs for information or to perform actions. These are also called tools.

AI Agents are a lot like “intelligent” functions

Has inputs & outputs.

- **Functions** take arguments and can (optionally) return something,
- **AI agents** process prompts or data and then generate responses or actions.

Encapsulates logic

- **Functions** contain code that executes a specific task.
- **AI agents** encapsulate reasoning steps and decision-making processes.

Modular & reusable

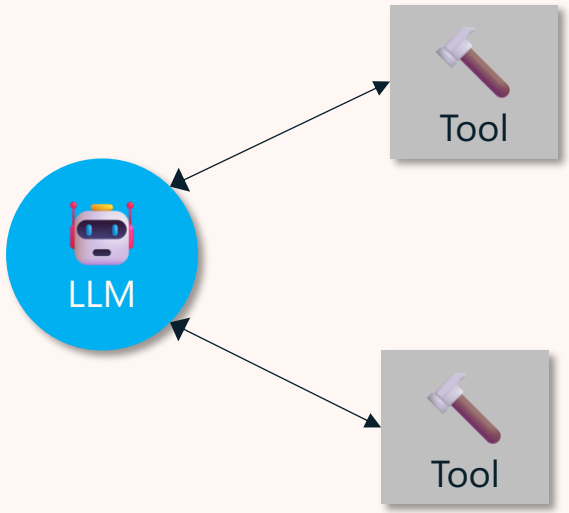
- **Both Functions** and **AI agents** can be “called” repeatedly in different parts of an application each time performing their specialized task.

Composable

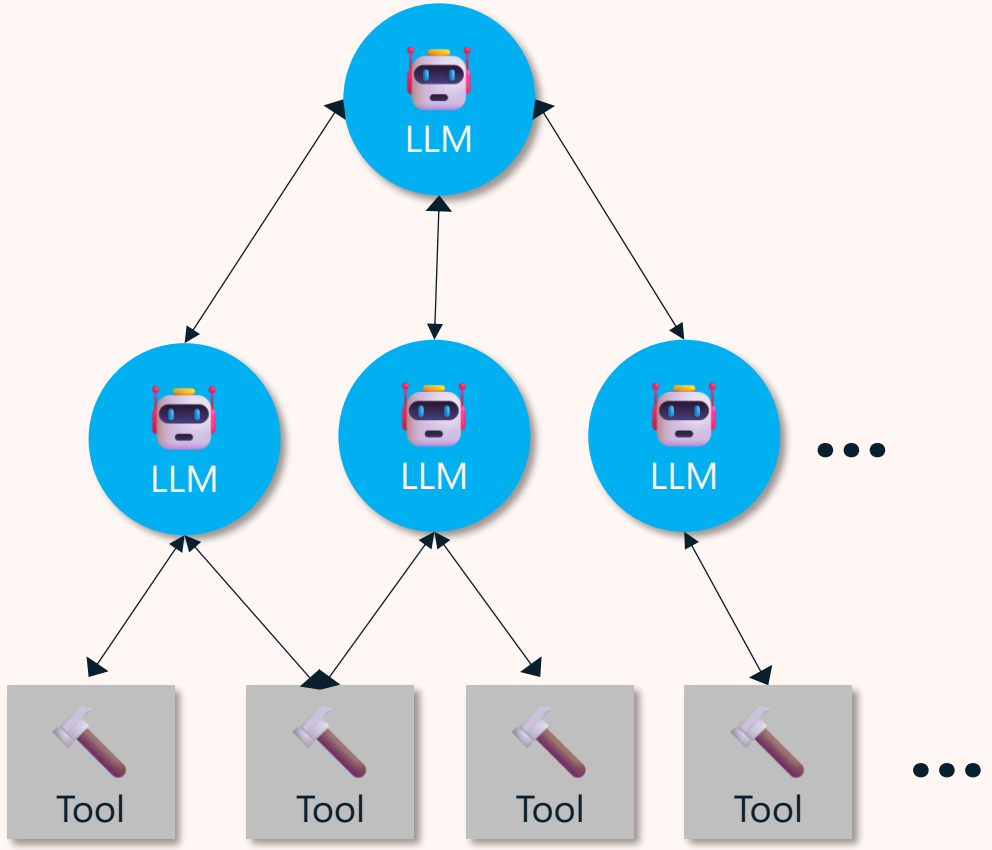
- Both **Functions** and **AI Agents** and can be chained or orchestrated with other agents/functions, enabling complex workflows from simpler building blocks.

Two popular agentic architectures

Single-Agent



Router (Supervisor, Planner) Multi-agent



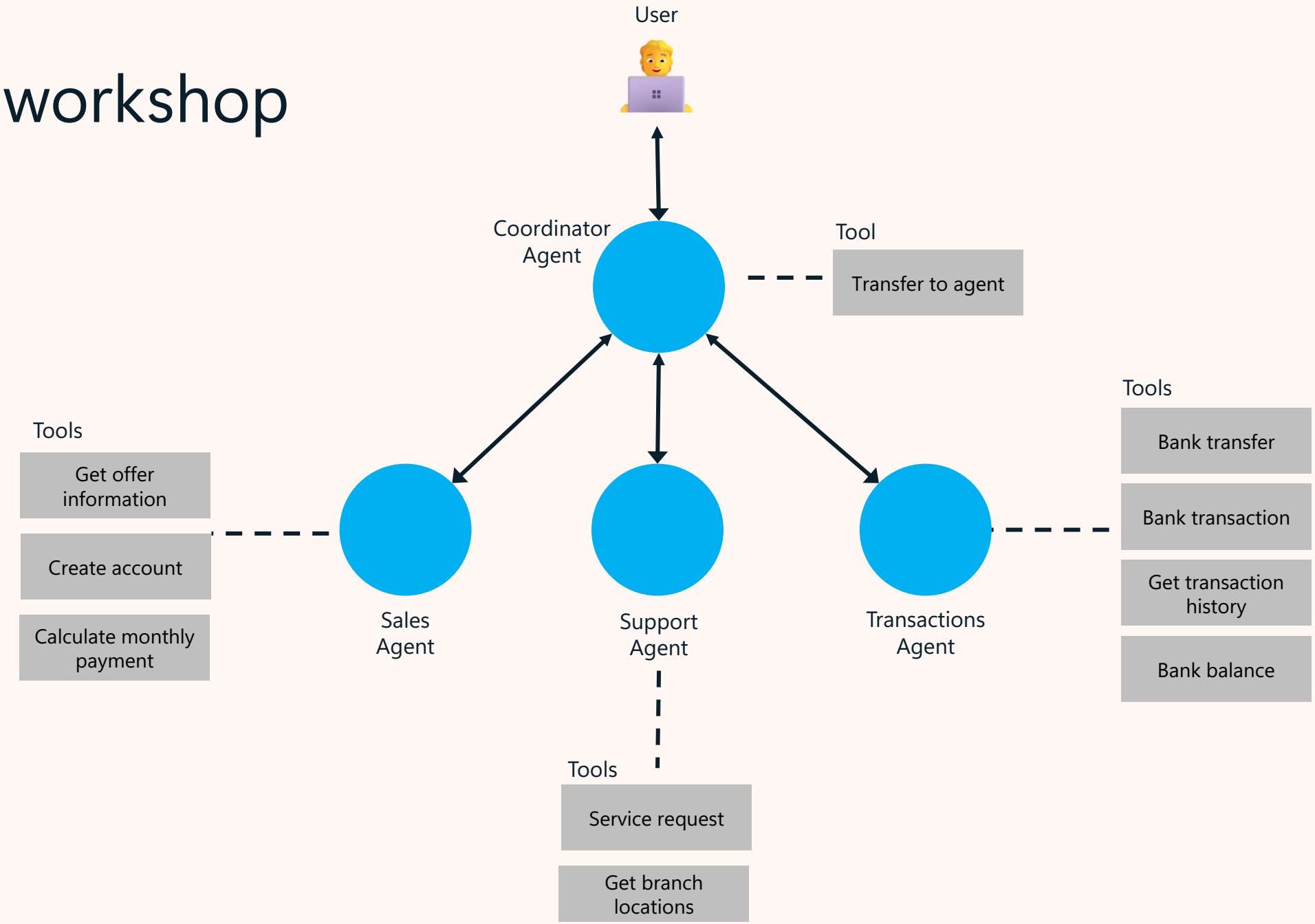
Why multi-agent?

Multi-agent applications utilize multiple specialized LLMs working collaboratively to address complex tasks.

Specialized agents can handle distinct tasks leading to more deterministic and accurate outcomes compared to a single, generalist LLM.

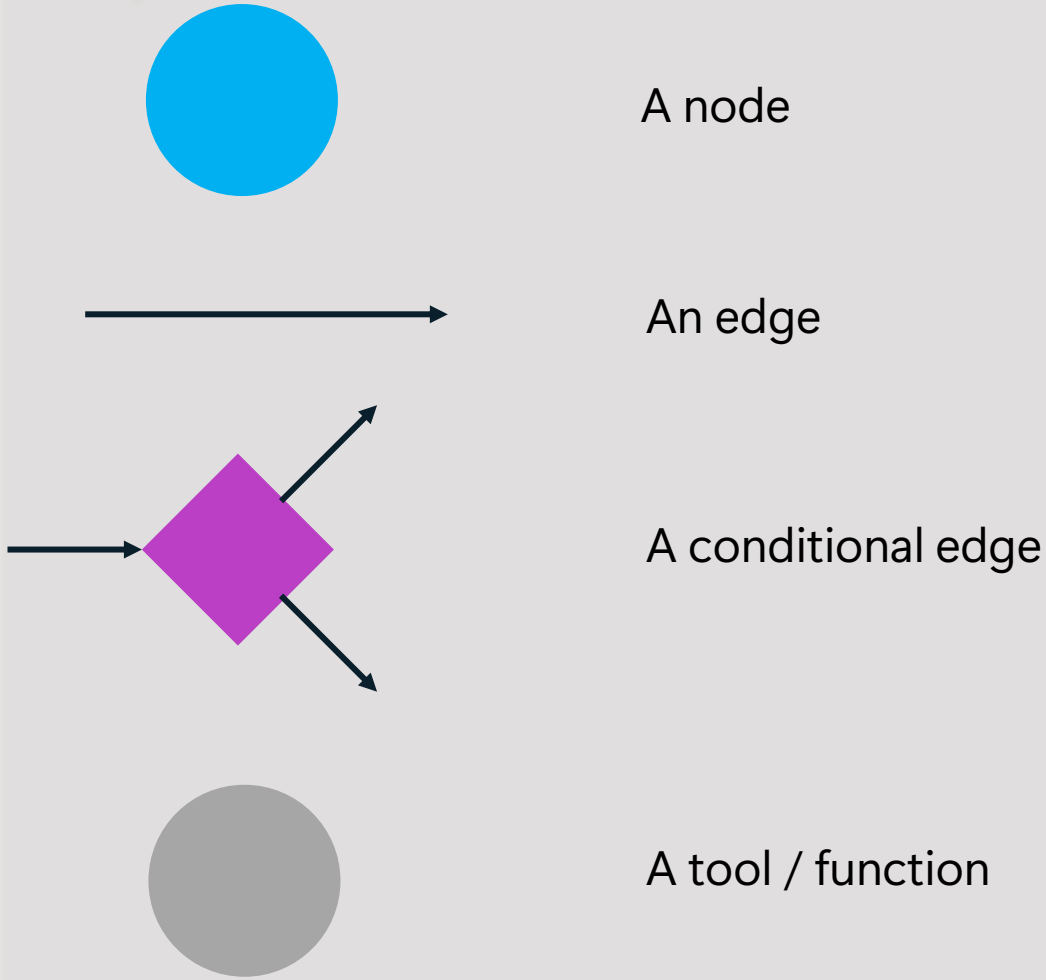
By assigning specific roles to different LLM agents, such as data retrieval, analysis, tool usage, etc., they can communicate and coordinate to solve problems more effectively and predictably.

This workshop

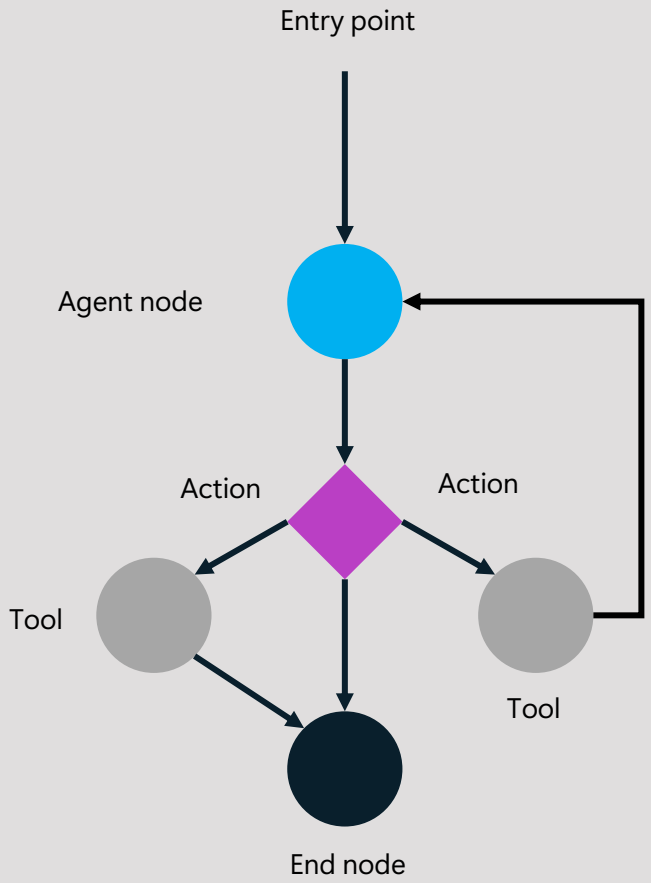


LangGraph - Graphs

Components



Example



Sources:

- https://langchain-ai.github.io/langgraph/concepts/low_level/#graphs
- https://learn.deeplearning.ai/courses/ai-agents-in-langgraph

LangGraph - Tools

Tools are an easy way to associate Python functions with a schema defining the functions name, description, and arguments.

These can be invoked by a model, and outputs can be fed back into the model.

Some outputs can be “artifacts”, or objects that can’t be directly passed to a model but need to be passed to downstream tasks.

Tips

- Design tools with clear naming, documentation, proper type hints to facilitate ease of use by models.
- Develop simple or narrowly scoped tools to ensure accurate and efficient model utilization.

Example code:

```
from langchain_core.tools import
tool

@tool

def multiply(a: int, b: int) -> int:
  """Multiply two numbers.""“

return a * b
```

State & Memory

State

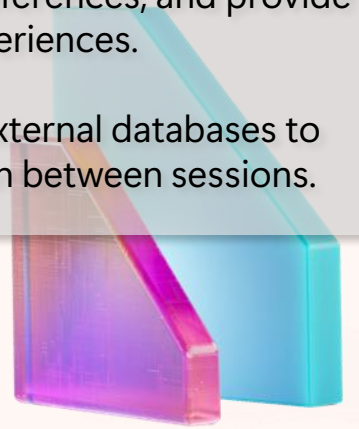
- A snapshot of your agentic application.
- It can contain any information useful to describe the snapshot.
- Ideally, it should have enough information such that the application can be restored at a given state.
- It can also help maintain consistency and context, enabling agents to perform tasks effectively and coordinate with each other.

Short -Term Memory

- Storage of information within a single session or interaction, allowing agents to maintain context during that session.
- Enables coherent and contextually relevant responses by remembering recent interactions.

Long-Term Memory

- Persistent storage of information across multiple sessions, enabling agents to recall past interactions and user preferences over time.
- Allows agents to learn from feedback, adapt to user preferences, and provide personalized experiences.
- Integrates with external databases to retain information between sessions.



Why Cosmos DB for AI Apps ?

GenAI + Data Use Cases with Azure Cosmos DB

Operational + Vector Database

No ETL

Consistent data

Reduce complexity & costs

Retrieval Augmented Generation (RAG)

Personalize LLM on your data

Cheaper than fine tuning

Faster iteration on new data

Conversational History

Conversational context

UX improvements

LLM optimizations

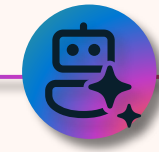
Auditing

Semantic Caching

Drastically reduces latency

Saves on Token consumption

Reduces costs and latency for LLM



AI Agents

Agent-to-agent transactions

Statefulness

Logging






Introducing langchain-azure-cosmosdb

Build AI Agents and RAG with One Database

A unified Python connector for LangChain and LangGraph that turns Azure Cosmos DB for NoSQL into the single persistence layer for all agentic app scenarios.

Why a Dedicated Connector?

Building AI agents today means stitching together half a dozen services — a vector database, a chat history store, a checkpointer, a semantic cache, a long-term memory layer. Each adds operational overhead, latency, and technical debt.

One Database, One Package

langchain-azure-cosmosdb collapses that stack into a single, highly scalable source of truth. It ships six integrations with both sync and async variants, plus support for access key and Managed Identity (Entra ID) authentication.

Install

```
pip install langchain-azure-cosmosdb
```

Introducing langchain-azure-cosmosdb: Build Agentic Apps and RAG with One Database - Azure Cosmos DB Blog

Source: devblogs.microsoft.com/cosmosdb/langchain-azure-cosmos-db-agents-rag/

Six Integrations, One Database

Vector Store AzureCosmosDBNoSqlVectorSearch — Vector, full-text (BM25), hybrid (vector+text with RRF), and weighted hybrid search powered by DiskANN indexes.

Semantic Cache AzureCosmosDBNoSqlSemanticCache — Cache LLM responses to reduce latency and cost on repeated or similar queries.

Chat History CosmosDBChatMessageHistory — Persist conversation history with configurable TTL support.

LangGraph Checkpointer CosmosDBSaver / CosmosDBSaverSync — Persist LangGraph graph state per thread_id across invocations for multi-turn agent memory.

LangGraph Cache CosmosDBCache / CosmosDBCacheSync — Node-level result caching for LangGraph workflows.

LangGraph Store CosmosDBStore / AsyncCosmosDBStore — Long-term memory with namespace organization and optional vector search.

Available on PyPI: pip install langchain-azure-cosmosdb

LangGraph - Checkpointers

Easy state management. Capture and store the current state of agents, including the data they're handling, their internal status, and relevant context. This simplifies tracking the progress and condition of your agents.

Reliable persistence. Save states securely to durable storage (like databases or file systems). This means your application can easily recover its previous state after restarts or system crashes, minimizing downtime and ensuring reliability.

Effortless restoration. Restore agents to previously saved checkpoints. This feature allows applications to resume smoothly from specific, stable points without reprocessing earlier steps.

Consistency across agents. In distributed or multi-agent systems, checkpointers ensure consistent states by coordinating checkpoints across different agents, even when they operate in separate nodes or environments.

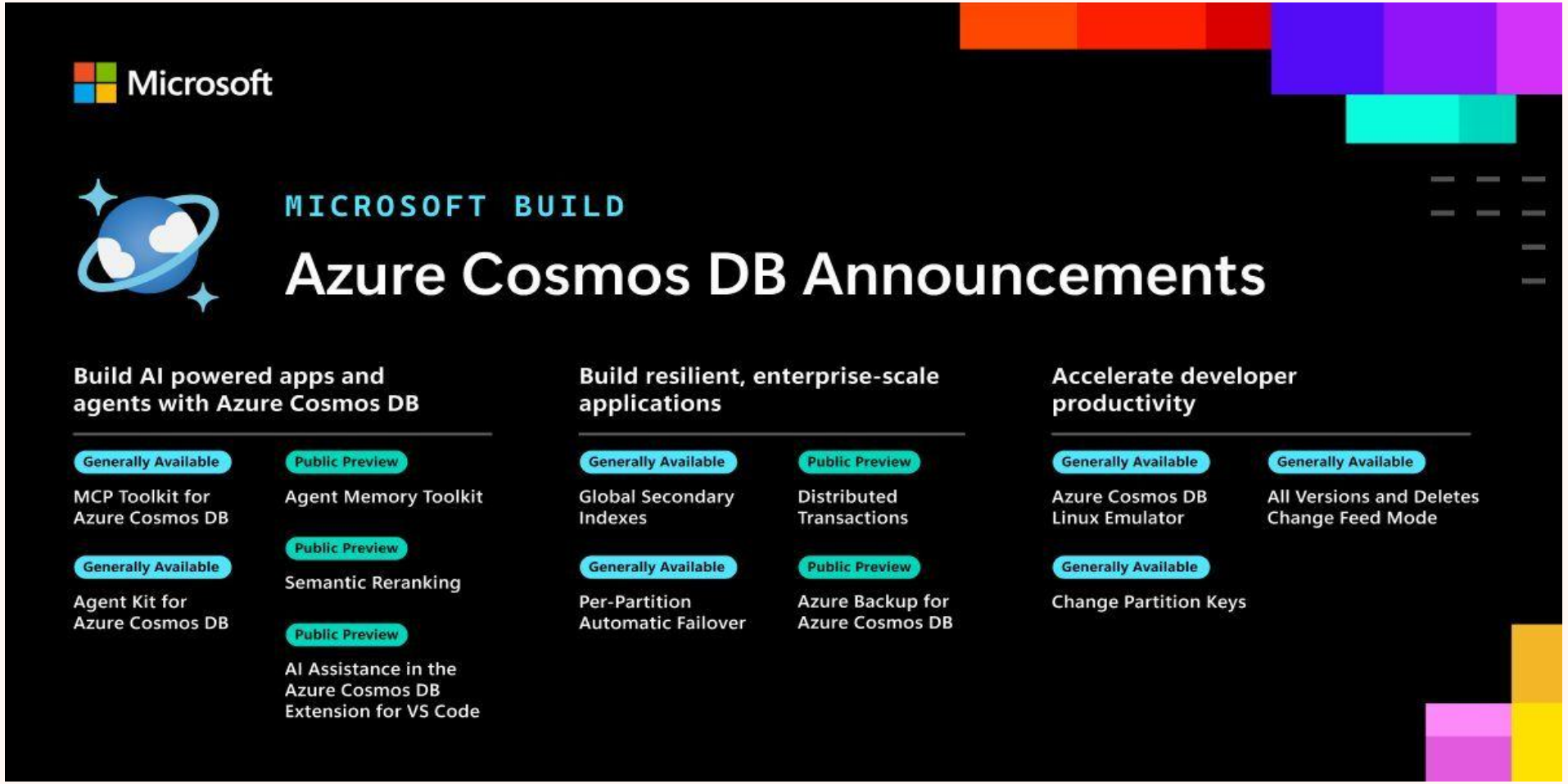
Customizable configuration. Developers can control how often and under what conditions checkpoints are created. This flexibility helps balance performance and reliability based on specific needs.

Built-in memory. Enable agents to "remember" past interactions, essential for ongoing conversations or repetitive tasks. This persistent memory maintains context across multiple interactions, enhancing user experience and interaction continuity.

Time travel & debugging. Allowing developers to replay past executions to review, debug, or analyze specific interactions. Additionally, developers can fork the graph state at any checkpoint, exploring alternative scenarios or debugging different pathways.

Azure Cosmos DB Checkpointer: https://github.com/shengbo-ma/langgraph-checkpoint-cosmos

Keep up with Azure Cosmos DB pace of innovation (announced at Build): What does it mean for you?



Announced at MS Build 2026: Azure Cosmos DB MCP Toolkit, Semantic Reranking, Global Secondary Indexes, and more! - Azure Cosmos DB Blog

New features in vector search

Float16 vectors

Save on 50% on storage while keeping recall high

Generally Available

Performance improvements

Faster vector inserts, indexing, and search

Modifiable vector indexes

Drop and add vector policies and indexes without creating new containers

Coming early 2026

Public preview of Semantic Reranker

AI-powered re-ranking of query results

Public Preview: Integrated Embeddings for Azure Cosmos DB

Automatically generate and maintain vector embeddings for your data

- **Built-in embedding generation** – Create and update embeddings automatically as data changes
- **No custom pipelines required** – Eliminate integration code, change tracking, and embedding orchestration
- **Always up to date** – Embeddings stay synchronized with source data automatically
- **Accelerate AI development** – Reduce time and complexity when building RAG, search, and agent applications
- **Integrated with Microsoft Foundry** – Supports Azure OpenAI embedding models through Microsoft Foundry

Announcing Public Preview at Build 2026

Full-Text and Hybrid Search in Azure Cosmos DB

- **Built-in relevance search** using BM25 ranking – no external search service required
- **Search across text fields** in JSON documents for keywords and phrases
- **Language-aware indexing**
- **Always up-to-date** – new or modified data becomes searchable in real time

- **Hybrid Search** combines semantic and keyword search for **more accurate, context-aware results**
- **Optimized for agentic app scenarios**

Common use cases for Full-Text Search

- E-commerce and retail
- Content management and publishing
- Customer support & knowledge bases
- Social and community apps

Common use cases for Hybrid Search

- Retrieval-Augmented Generation (RAG)
- Intelligent recommendation systems
- Enterprise document intelligence
- Multi-modal AI applications

New features in full-text search

Full-text: Fuzzy search

Approximate & typo-resilient text search

Generally Available

Full-text: New languages

German, French, Spanish and now Italian, Portuguese, and Brazilian Portuguese

Public Preview

Full-text: Faster Inserts & BM25 Scoring

Lower RU charges, lower latencies on indexing and phrase searches

Full-text: Custom stop words

Flexible and custom stop words increases search relevancy

New- 2026

Public Preview: Semantic Reranker in Azure Cosmos DB

Improve search and query relevance with AI-powered ranking

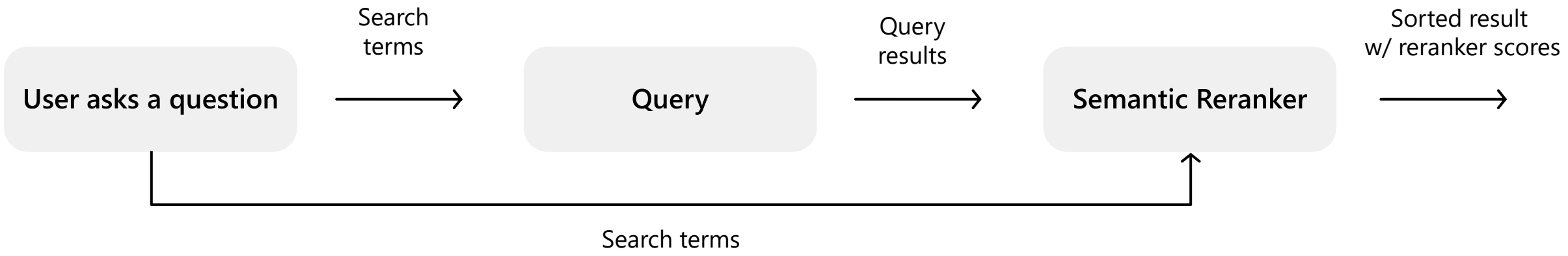
- **More relevant results** – Reorders results based on meaning and context, not just keyword matches
- **Works with existing searches** – Enhances vector, full-text, and hybrid search workloads
- **Better AI application quality** – Improves grounding for RAG, copilots, agents, and question-answering systems
- **No complex ranking pipelines** – Eliminates the need for custom scoring and manual relevance tuning
- **Integrated experience** – Add semantic relevance directly within Azure Cosmos DB

Announcing Public Preview at Build 2026

Semantic Reranking in Azure Cosmos DB

Public Preview

Use AI to improve search relevancy



Featuring the world-class **Microsoft AI Semantic Ranker model** available today in Azure AI Search

Sign-up today! https://aka.ms/AzureCosmosDB/RerankerPreview

MCP Toolkit for Azure Cosmos DB

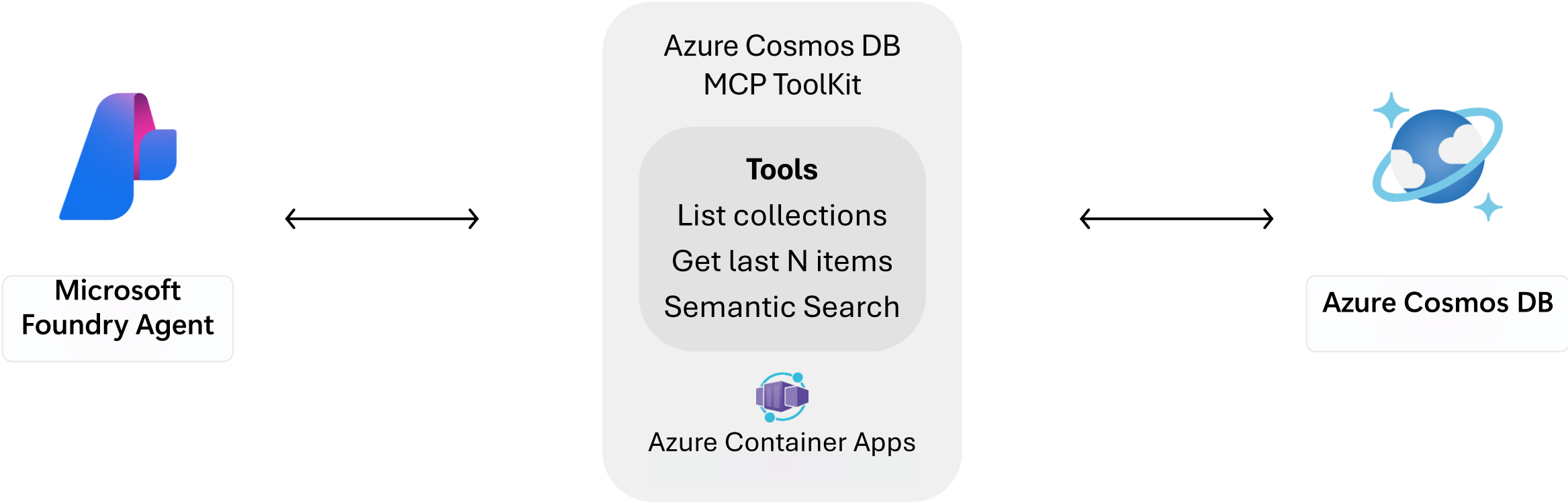
Connect AI agents to operational data with Model Context Protocol

- **Standardized agent integration** – Expose Azure Cosmos DB capabilities through MCP
- **Accelerate agent development** – Reduce custom integration code and simplify tool connectivity
- **Enterprise-ready foundation** – Designed for secure, reliable, and scalable agentic applications
- **Operational data for AI agents** – Enable agents and copilots to interact directly with Azure Cosmos DB data
- **From prototype to production** – Build and deploy data-aware AI experiences with greater confidence

Announcing General Availability at Build 2026

MCP Toolkit for Azure Cosmos DB

First Class support with Microsoft Foundry Agents Service and Azure Cosmos DB



Agent Memory Toolkit for Azure Cosmos DB

Build scalable memory systems for AI agents faster

- **Simplified agent memory architecture** – Store, update, and retrieve memories through a simple developer interface
- **Accelerate development** – Eliminate the need to build custom memory pipelines from scratch
- **Improved retrieval relevance** – Built-in memory processing and retrieval orchestration
- **Scalable and durable foundation** – Designed for production-ready agent and copilot applications
- **Focus on application logic** – Reduce infrastructure complexity and service integration effort

Announcing Public Preview at Build 2026

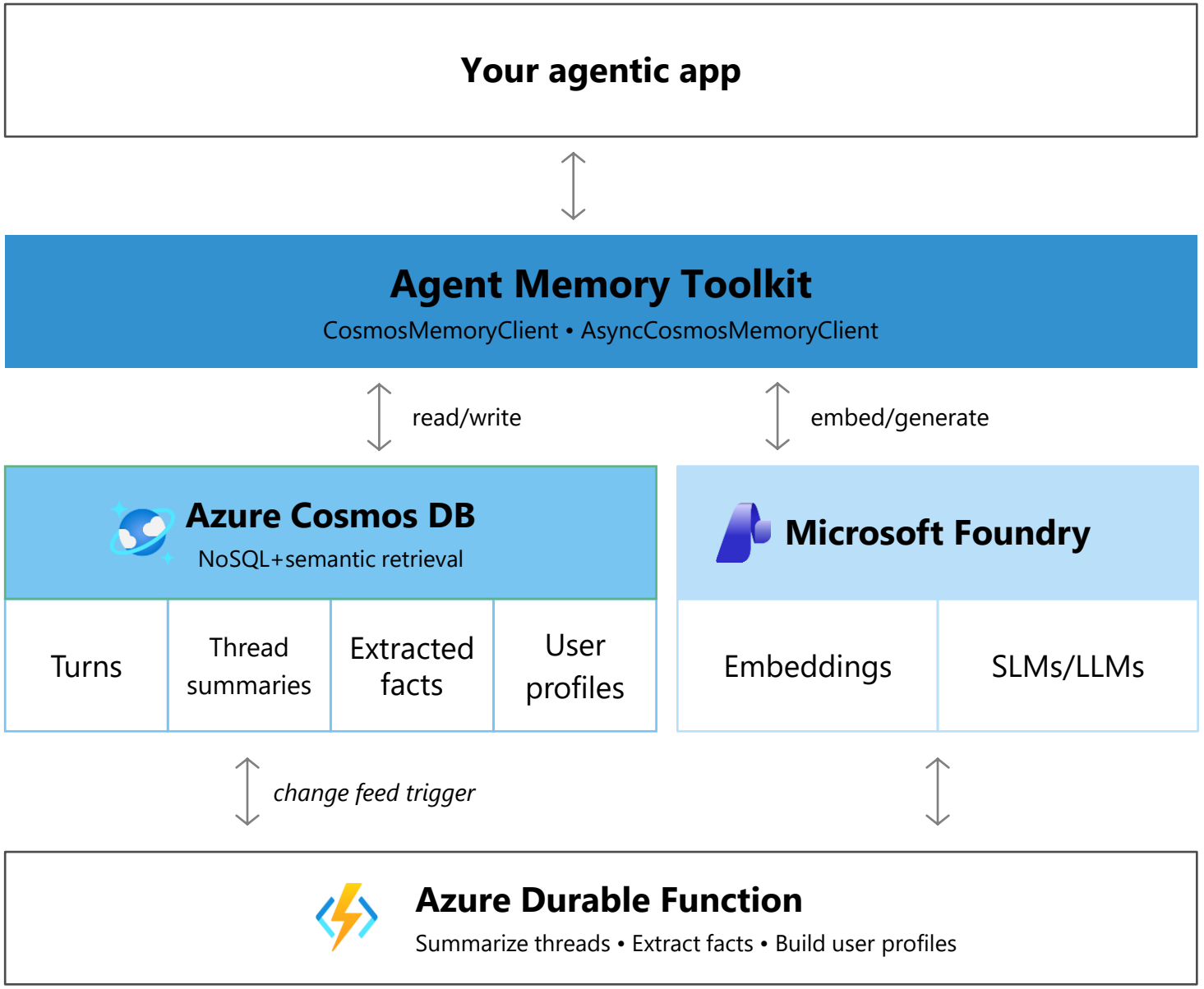
Azure Cosmos DB

Agent Memory Toolkit

A Python SDK & serverless architecture

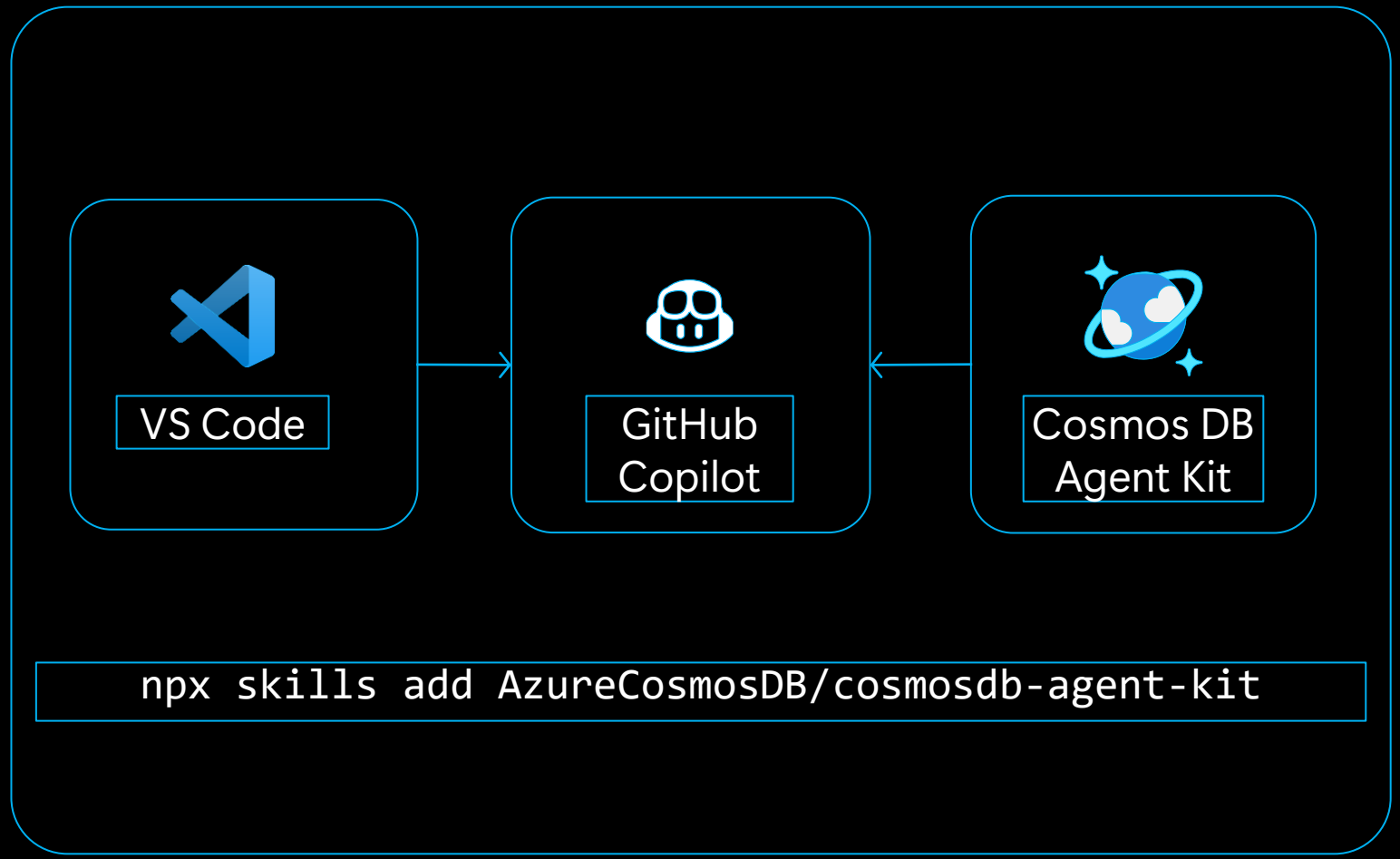
Store, process, and retrieve memories automatically

Facts, summaries, procedural, episodic, & more



Azure Cosmos DB Kit - GA

Azure Cosmos DB Agent Kit



Skills

- Data Modeling
- Partition Keys
- Queries
- SDK Best Practices
- Indexing
- Throughput
- High Availability
- Monitoring

https://aka.ms/azurecosmosdb-agent-kit

Generally Available: Agent Kit for Azure Cosmos DB

Help AI coding agents build optimized Cosmos DB applications

- **Built-in Cosmos DB expertise** – More than 100 best-practice rules for data modeling, performance, and scale
- **Smarter AI-assisted development** – Guides architectural, query, and partitioning decisions in real time
- **Prevent costly mistakes** – Avoid hot partitions, inefficient queries, and scaling bottlenecks
- **Accelerate developer productivity** – Build production-ready applications faster with AI-powered guidance
- **Works where developers work** – Integrates with Visual Studio Code, GitHub Copilot, and AI coding environments

Announcing General Availability at Build 2026

Public Preview: AI Assistant in the Azure Cosmos DB VS Code Extension

Use AI to explore, query, and manage Azure Cosmos DB

- **Natural language to query** – Generate optimized Azure Cosmos DB queries using plain English
- **AI-powered query assistance** – Explain, refine, and troubleshoot queries directly in Visual Studio Code
- **Intelligent development experience** – Get contextual auto-completion for queries and code
- **Agent-powered database operations** – Use Azure Cosmos DB Shell with MCP to navigate and interact with databases using AI agents
- **Boost developer productivity** – Spend less time on syntax and more time building applications

Announcing Public Preview at Build 2026